ARMIS Platform

Architecture Document — v2.4

1. Overview

ARMIS follows a microservices architecture where each service owns its data, is independently deployable, and communicates with other services through well-defined interfaces. Services do not share databases. The API Gateway is the single entry point for all external traffic.

2. Request Flow

Every external request follows this path:

1. Client sends HTTPS request to API Gateway (port 8080)
2. API Gateway validates the JWT token via auth-service
3. If valid, gateway forwards the request to the appropriate downstream service over gRPC (internal network)
4. Downstream service processes the request, reads/writes its own PostgreSQL schema
5. Response travels back through the gateway to the client

For event-driven flows (e.g., mission status change triggering notifications):

6. mission-service publishes an event to RabbitMQ exchange armis.events
7. notification-service consumes the event from its queue armis.notifications
8. notification-service sends push notification or email

3. Service Descriptions

3.1 API Gateway

Built on Spring Cloud Gateway. Responsibilities:

- TLS termination (certificates managed by cert-manager in Kubernetes)
- JWT validation — delegates to auth-service /validate endpoint
- Rate limiting — 100 req/min per user token, stored in Redis
- Request routing via route predicates defined in application.yml
- CORS policy enforcement

The gateway does NOT perform business logic. If you need to add a new route, edit api-gateway/src/main/resources/application.yml.

3.2 Auth Service

Handles all identity and access management. Uses Spring Security with a custom JWT provider.

- Users are stored in the users table in the auth schema
- Roles: ROLE_ADMIN, ROLE_OPERATOR, ROLE_VIEWER
- Tokens expire after 8 hours; refresh tokens expire after 7 days
- Passwords are hashed with BCrypt (strength 12)
- Failed login attempts are tracked in Redis; account locks after 5 failures in 10 minutes

3.3 Asset Service

Manages the registry of tracked assets (vehicles, personnel, equipment) and their real-time locations.

- Asset metadata (name, type, owner unit) is stored in PostgreSQL
- Real-time location updates are received via WebSocket connections from field devices
- Location history is stored in a TimescaleDB hypertable for efficient time-series queries
- The /assets/{id}/location endpoint returns current position; /assets/{id}/track returns history

3.4 Mission Service

Core business logic for mission planning and execution tracking.

- Mission lifecycle: DRAFT -> APPROVED -> ACTIVE -> COMPLETED | CANCELLED
- State transitions are enforced via a state machine (Spring State Machine)
- Only ROLE_ADMIN and ROLE_OPERATOR can create or approve missions
- ROLE_VIEWER can only read mission data
- On status change, service publishes MissionStatusChangedEvent to RabbitMQ

3.5 Notification Service

Stateless consumer of RabbitMQ events. Does not expose REST endpoints (internal only).

- Listens on armis.notifications queue
- Sends push notifications via Firebase Cloud Messaging (FCM)
- Sends email via SMTP (configured in notification-service/src/main/resources/application.yml)
- Failed deliveries are retried 3 times with exponential backoff, then moved to dead-letter queue

3.6 Report Service

Generates async reports (PDF, Excel) on demand. Reports are generated in background threads and uploaded to S3.

- Client POSTs a report request; service returns a reportId immediately
- Client polls GET /reports/{reportId}/status until status is COMPLETED
- Download URL is a pre-signed S3 URL valid for 1 hour
- Report templates are Jasper Reports (.jrxml) files in report-service/src/main/resources/reports/

4. Database Strategy

Each service has its own PostgreSQL schema within the shared PostgreSQL cluster (dev/staging) or dedicated instance (production). Cross-service joins are forbidden. If data from another service is needed, use the service's REST/gRPC API.

Service	Schema	Key Tables
auth-service	auth	users, roles, refresh_tokens
asset-service	assets	assets, asset_locations (hypertable)
mission-service	missions	missions, mission_assignments, mission_events
report-service	reports	report_requests, report_results

5. Security Architecture

5.1 Authentication Flow

1. Client POSTs credentials to POST /auth/login via API Gateway.

2. auth-service validates credentials, returns {accessToken, refreshToken}.

3. Client includes accessToken as Bearer token in all subsequent requests.

4. API Gateway calls auth-service GET /auth/validate on every request.

5. If token is expired, client uses refreshToken to obtain a new accessToken via POST /auth/refresh.

5.2 Service-to-Service Security

In production (Kubernetes + Istio), all service-to-service communication uses mutual TLS (mTLS) enforced by the Istio service mesh. Each service has an Istio sidecar proxy. In local development, services communicate over plain localhost without mTLS.

6. Scalability Considerations

- Stateless services (api-gateway, auth, mission, notification, report) can be horizontally scaled by adding replicas in Kubernetes
- asset-service WebSocket connections are sticky-session routed via Istio to maintain WebSocket state
- Redis Cluster is used in production for cache scalability
- RabbitMQ uses a 3-node cluster in production for HA

7. Known Limitations & Technical Debt

- report-service uses synchronous Jasper rendering on the main thread pool — under high load this causes latency spikes. ADR-004 proposes migrating to a dedicated worker pool.
- auth-service does not yet support LDAP/SSO integration. Planned for Q3 2026.
- TimescaleDB (asset-service) requires a separate extension install; documented in ONBOARDING.md.